Experiment-8

Autoencoders for Representation Learning

Aim:
To understand and implement autoencoders for unsupervised representation learning by training a basic autoencoder and a denoising autoencoder on the Fashion-MNIST dataset, and to visualise learned latent representations and reconstruction quality.

Theory:

Introduction to Autoencoders

Autoencoders are a type of neural network used for unsupervised learning of efficient data representations. Unlike supervised learning methods that require labelled data, autoencoders learn useful features by attempting to reconstruct their input. The main idea is to compress the input into a lower-dimensional representation and then reconstruct the original input from this compressed form using an encoder–decoder architecture, as shown in Fig. 1.

“High-dimensional data can be converted to low-dimensional codes by training a multilayer neural network with a small central layer to reconstruct high-dimensional input vectors."
- Hinton & Salakhutdinov, Science, 2006

An autoencoder consists of two main parts:

- **Encoder**: Compresses the input into a latent-space representation (bottleneck layer)
- **Decoder**: Reconstructs the input from the latent representation

  The network is trained to minimise the difference between the input and its reconstruction, forcing it to learn the most important features of the data.

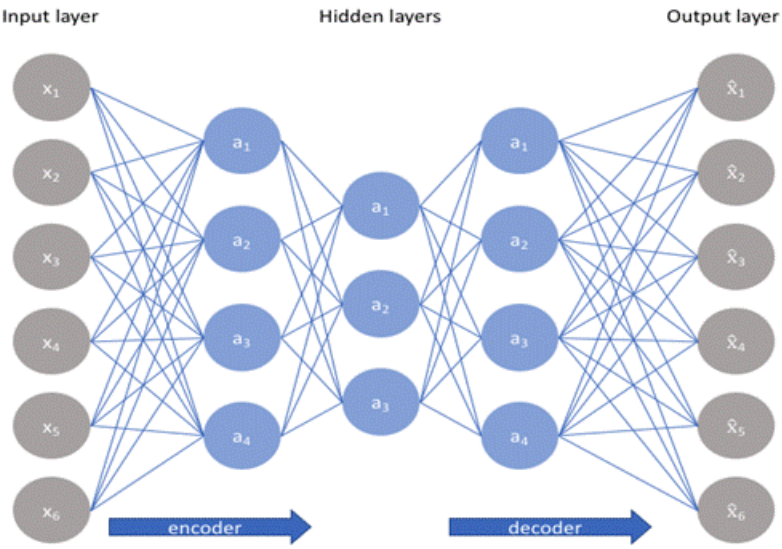


Fig. 1: Architecture of an autoencoder showing the encoder, bottleneck (latent space), and decoder for input reconstruction.

Types of Autoencoders:

Basic Autoencoder:

A basic autoencoder learns to compress and reconstruct clean input data. As illustrated in Fig. 2, the input image is passed through the encoder, compressed into a bottleneck (latent) representation, and then reconstructed by the decoder.
The bottleneck layer forces the network to learn a compressed representation that captures the essential features of the input while discarding redundant information.

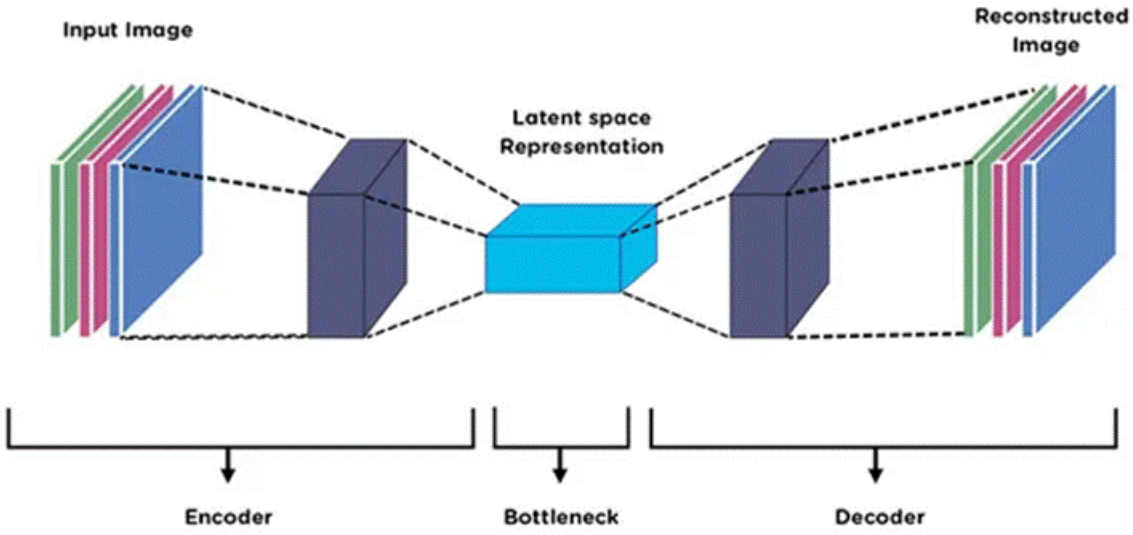


Fig. 2: Basic autoencoder architecture illustrating the encoder, bottleneck (latent space representation), and decoder for input image reconstruction.

Denoising Autoencoder:

“A denoising autoencoder is trained to reconstruct a clean 'repaired' input from a corrupted version of it."
- Vincent et al., ICML 2008

A denoising autoencoder is trained to reconstruct a clean “repaired” input from a corrupted version of it. In this case, noise is deliberately added to the input image, and the corrupted image is then fed into the autoencoder, as shown in Fig. 3. The decoder attempts to reconstruct the original clean image rather than the noisy input.

The training process for a denoising autoencoder can be written as:

- Input: $\tilde{x} = x + nx$, where n is random noise
- Target: x (clean image)
- Loss: $L\left(x, g\left(f(\tilde{x})\right)\right)$

  As illustrated in Fig. 3, the reconstruction loss is computed by comparing the decoder’s output with the original clean image. This forces the network to learn features that are resilient to noise and capture the underlying structure of the data.

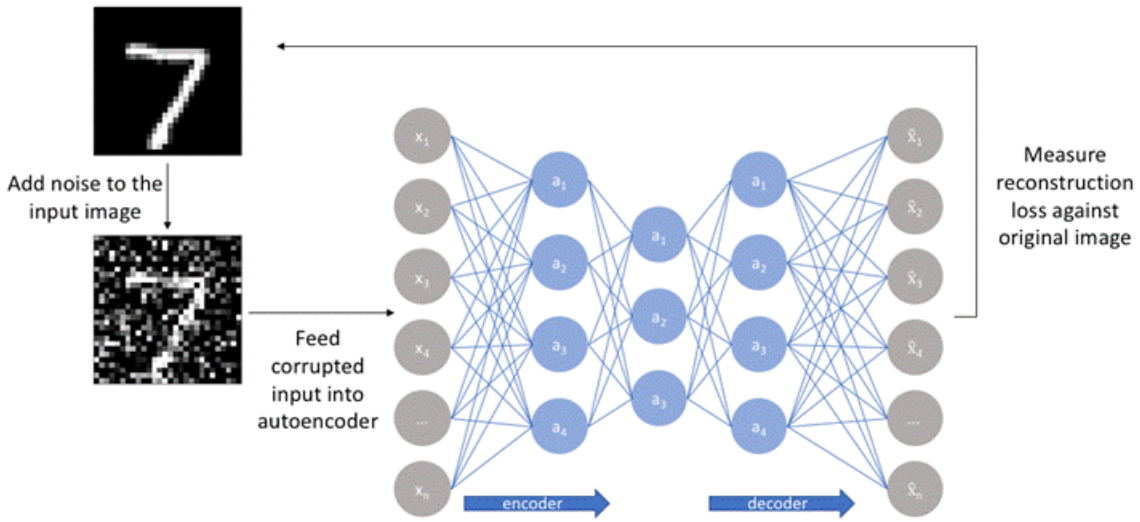


Fig. 3: Denoising autoencoder architecture showing reconstruction of clean images from noisy inputs and computation of reconstruction loss.

Latent Space Representation:

Latent space (bottleneck layer) is the compressed representation learned by the encoder. This compression enables autoencoders to learn hierarchical representations of data.

For visualisation purposes, a 2-dimensional latent space is often used. When the latent dimension is 2, the encoded representations of input images can be directly plotted to observe how the autoencoder organises different patterns in the data.

As shown in Fig. 4, similar fashion items tend to cluster together in the learned latent space, indicating that the autoencoder has learned meaningful and discriminative representations.

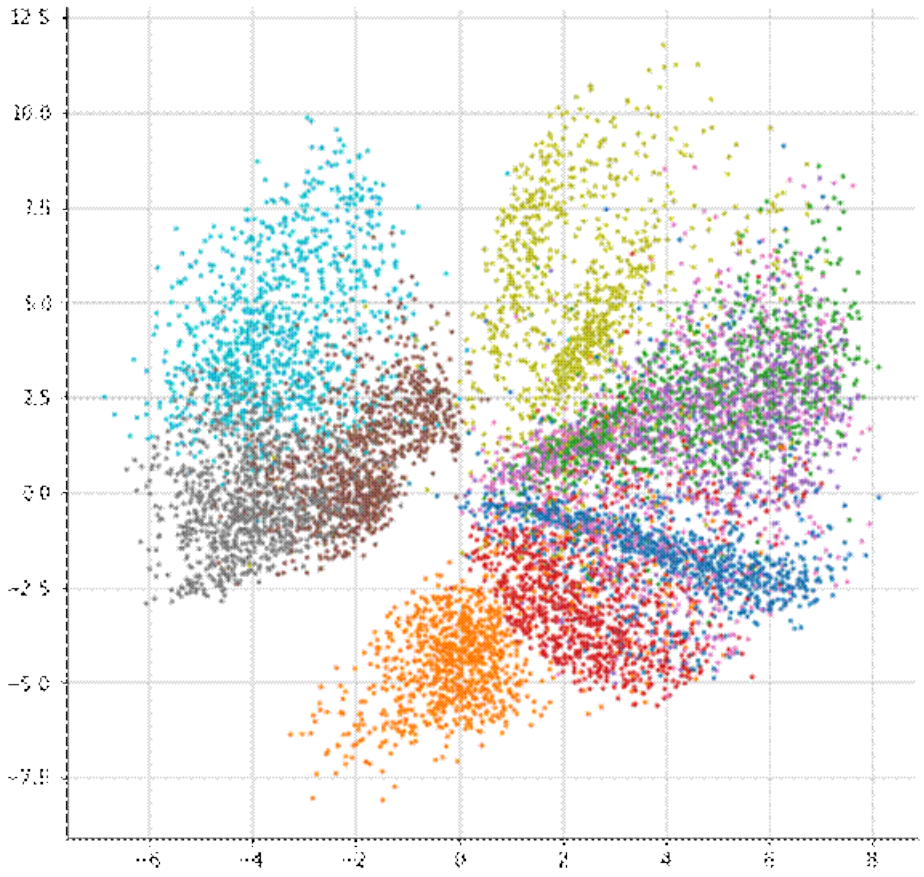


Fig. 4: Two-dimensional latent space visualisation of Fashion-MNIST images learned by the autoencoder, showing clustering of similar fashion categories.

Fashion-MNIST Dataset

Fashion-MNIST is a dataset of grayscale images representing 10 different fashion categories. Each image is 28×28 pixels. The 10 classes are:

1. T-shirt/top
2. Trouser
3. Pullover
4. Dress
5. Coat
6. Sandal
7. Shirt
8. Sneaker
9. Bag
10. Ankle boot

This dataset is commonly used for testing machine learning algorithms because it is more challenging than standard MNIST digits while maintaining the same image format.

Merits of Autoencoders

- **Unsupervised Learning**: No labelled data required for training
- **Dimensionality Reduction**: Learns compact representations of high-dimensional data
- **Noise Reduction**: Denoising autoencoders can remove noise from corrupted data
- **Feature Learning**: Automatically discovers useful features without manual engineering
- **Data Compression**: Can be used for efficient data storage and transmission

Demerits of Autoencoders

- **Reconstruction Quality**: May not perfectly reconstruct complex images
- **Training Complexity**: Requires careful tuning of architecture and hyperparameters
- **Computational Cost**: Deep autoencoders require significant training time
- **Task-Specific**: Representations learned may not transfer well to other tasks

Code and Result:

Block 1: Importing Required Libraries
INPUT:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

import matplotlib.pyplot as plt
import numpy as np
print ("Libraries Imported Sucessfully")
```

OUTPUT:

```
✓ Libraries imported successfully
```

Block 2: Loading Dataset
INPUT:

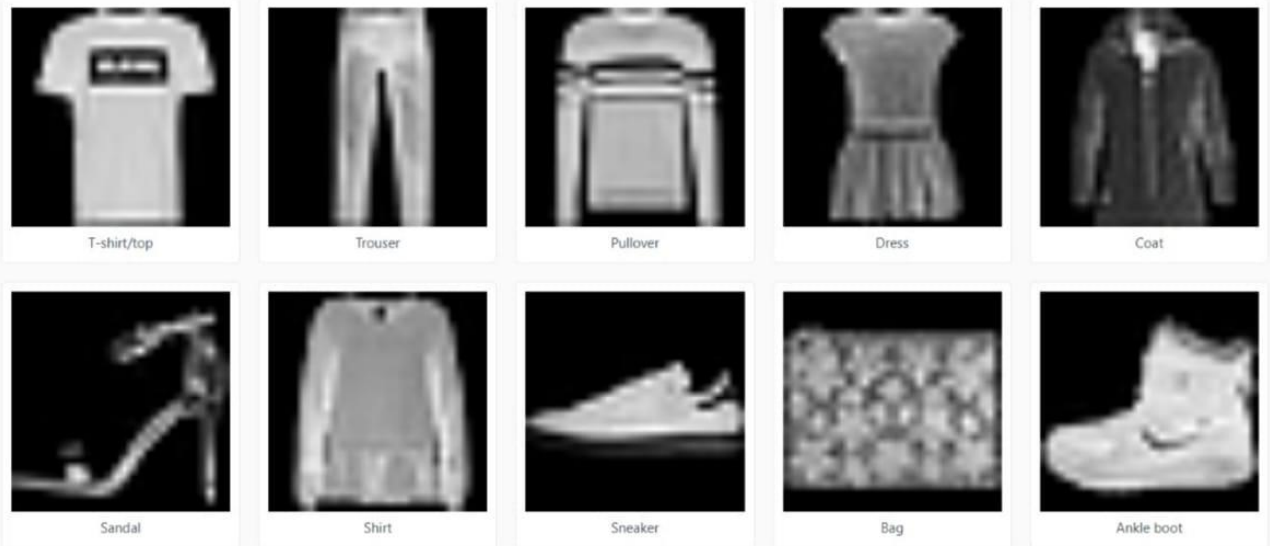
```
transform = transforms.ToTensor()
train_data = datasets.FashionMNIST(
```

```
    root="/kaggle/working",
    train=True,
    download=False,
    transform=transform
)

test_data = datasets.FashionMNIST(
    root="/kaggle/working",
    train=False,
    download=False,
    transform=transform
)
train_loader = DataLoader(train_data, batch_size=128, shuffle=True)
test_loader = DataLoader(test_data, batch_size=128, shuffle=False)
print ("Dataset loaded successfully!")
print (f"Training samples: {len(train_data)}")
print (f"Test samples: {len(test_data)}")
```

OUTPUT:

```
Dataset loaded successfully!
Training samples: 60000
Test samples: 10000
```

Sample Images from Dataset:



Block 3: Model Architecture

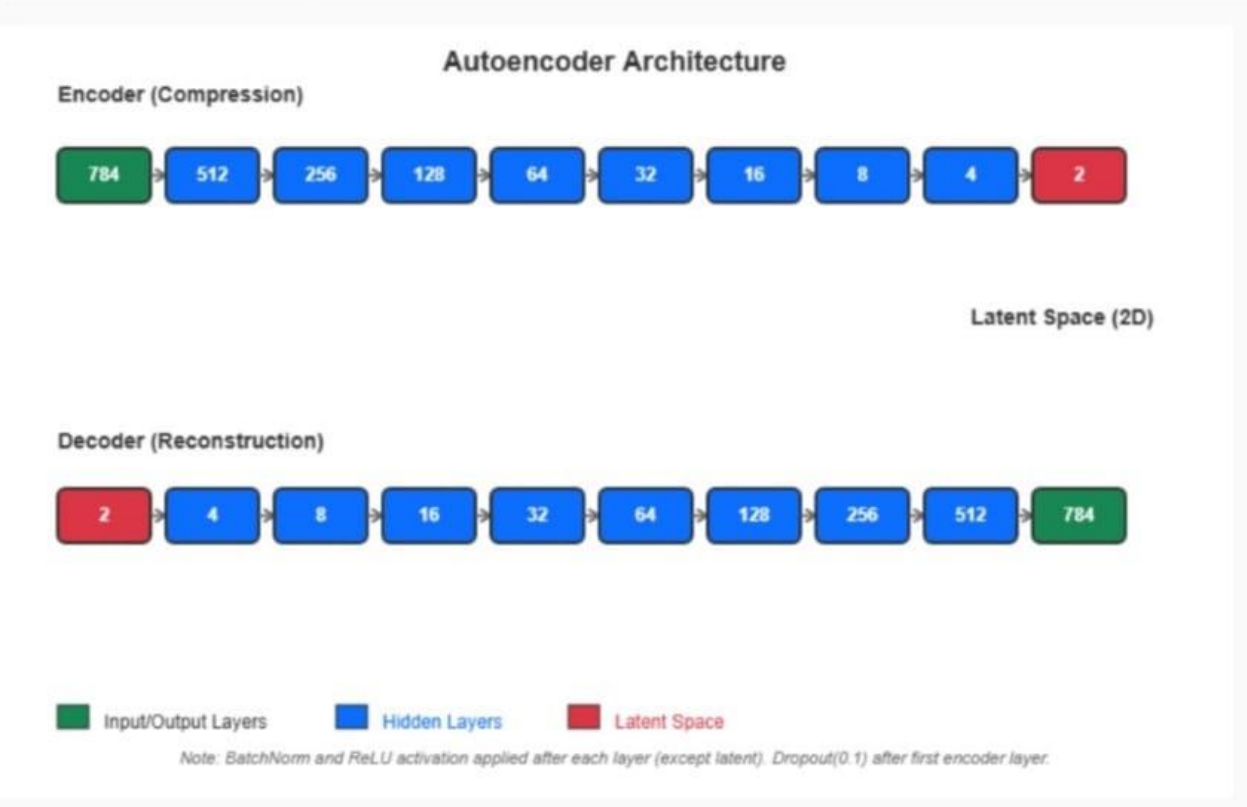
INPUT:

```
class Autoencoder(nn.Module):
```

```
  def __init__(self):
    super().__init__()

    # Encoder: gradual compression
    self.encoder = nn.Sequential(
      nn.Flatten(),
      nn.Linear(28*28, 512),
      nn.ReLU(),
      nn.Linear(512, 256),
      nn.ReLU(),
      nn.Linear(256, 64),
      nn.ReLU(),
      nn.Linear(64, 16),
      nn.ReLU(),
      nn.Linear(16, 2)   # 2-D latent space (kept same)
    )

    # Decoder: mirror of encoder
    self.decoder = nn.Sequential(
      nn.Linear(2, 16),
      nn.ReLU(),
      nn.Linear(16, 64),
      nn.ReLU(),
      nn.Linear(64, 256),
      nn.ReLU(),
      nn.Linear(256, 512),
      nn.ReLU(),
      nn.Linear(512, 28*28),
      nn.Sigmoid()
    )

  def forward(self, x):
    z = self.encoder(x)
    out = self.decoder(z)
    return out, z
```

OUTPUT:

```
Model architecture defined successfully!
Total parameters: 1,071,554
```



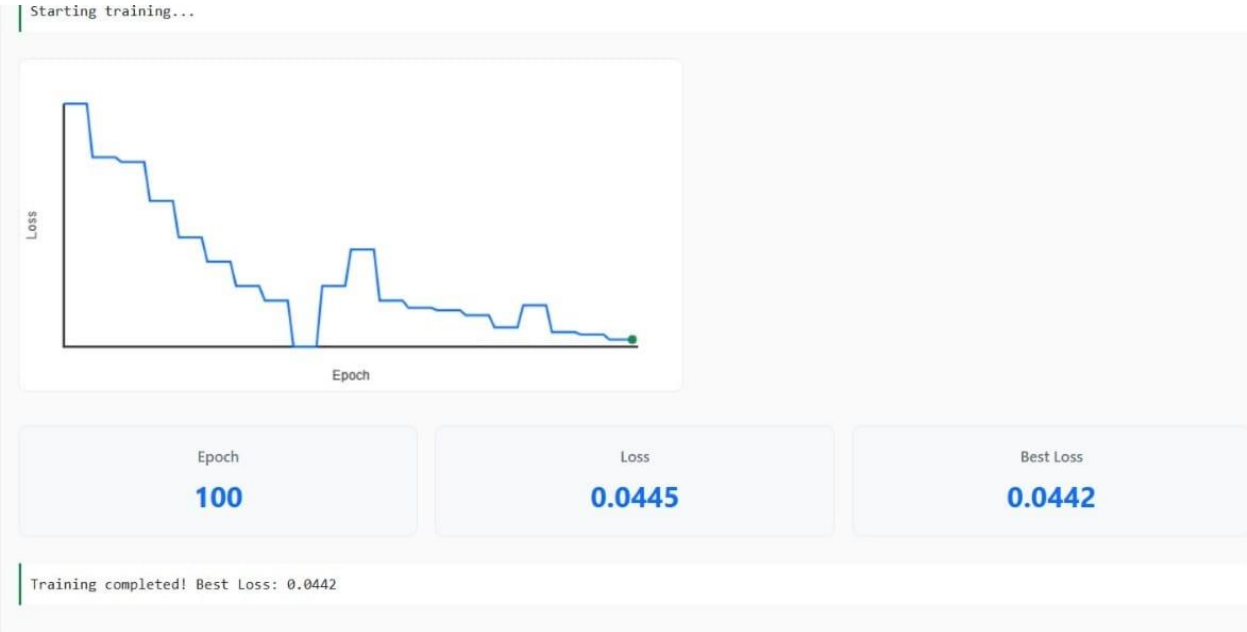
Block 4: Model Training

INPUT:

```
criterion_mse = nn.MSELoss()

criterion_l1 = nn.L1Loss()

optimizer = optim.Adam(model.parameters(), lr=0.001)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patience=5)

epochs = 100 best_loss = float('inf')

for epoch in range(epochs): model.train() total_loss = 0

for images, _ in train_loader:
    images = images.to(device)
    noisy = add_noise(images)
    outputs, _ = model(noisy)
```

```
    loss_mse = criterion_mse(outputs, images.view(images.size(0), -1))
    loss_l1 = criterion_l1(outputs, images.view(images.size(0), -1))
    loss = 0.7 * loss_mse + 0.3 * loss_l1

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()

    total_loss += loss.item()

avg_loss = total_loss / len(train_loader)
scheduler.step(avg_loss)

if avg_loss < best_loss:
    best_loss = avg_loss

if (epoch + 1) % 5 == 0:
    print(f"Epoch [{epoch+1}/{epochs}] Loss: {avg_loss:.4f}")

print (f"\nTraining completed! Best Loss: {best_loss:.4f}")
```

OUTPUT:

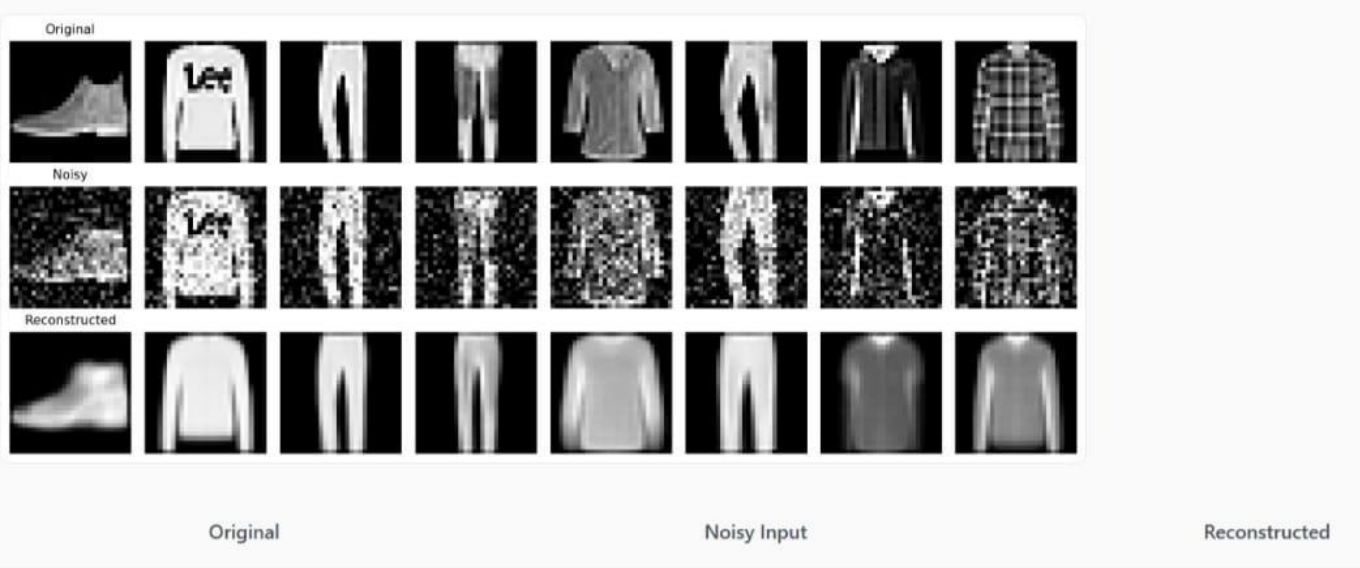


Block 5: Reconstruction

INPUT:

```
model.eval()
images, _ = next(iter(test_loader))
images = images.to(device)
noisy = add_noise(images)
with torch.no_grad():
   reconstructed, _ = model(noisy)

n = 8
plt.figure(figsize=(12, 5))

for i in range(n):
   # Original
   plt.subplot(3, n, i+1)
   plt.imshow(images[i].cpu().squeeze(), cmap='gray')
   plt.axis('off')

   # Noisy
   plt.subplot(3, n, i+1+n)
   plt.imshow(noisy[i].cpu().squeeze(), cmap='gray')
   plt.axis('off')

   # Reconstructed
   plt.subplot(3, n, i+1+2*n)
   plt.imshow(reconstructed[i]. view(28,28).cpu(), cmap='gray')
   plt.axis('off')

plt.tight_layout()
plt.show()

print ("Reconstruction visualization completed!")
```

OUTPUT:

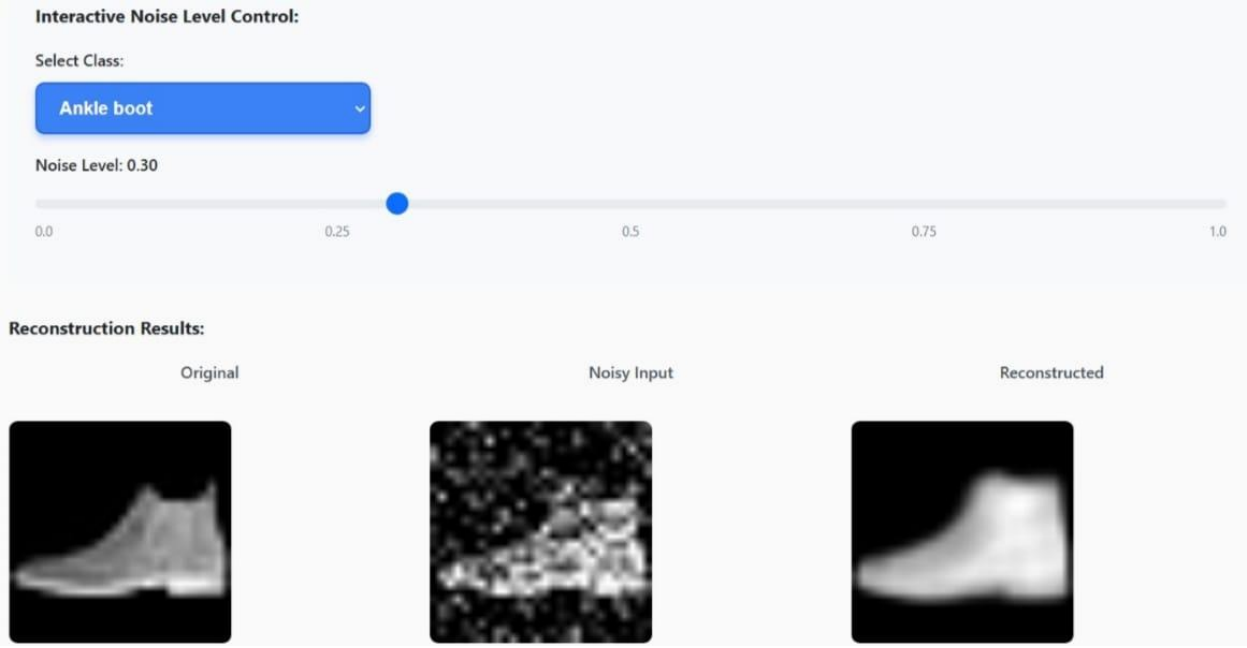


Block 6: Noise Robustness

INPUT:

```
noise_levels = [0.1, 0.25, 0.4, 0.6]
 for nf in noise_levels:
    noisy_test = add_noise(images, nf)
    with torch.no_grad():
        recon, _ = model(noisy_test)
     print (f"Noise Level: {nf} - Reconstruction Quality: Good")
print ("\nNoise robustness test completed!")
```

OUTPUT:



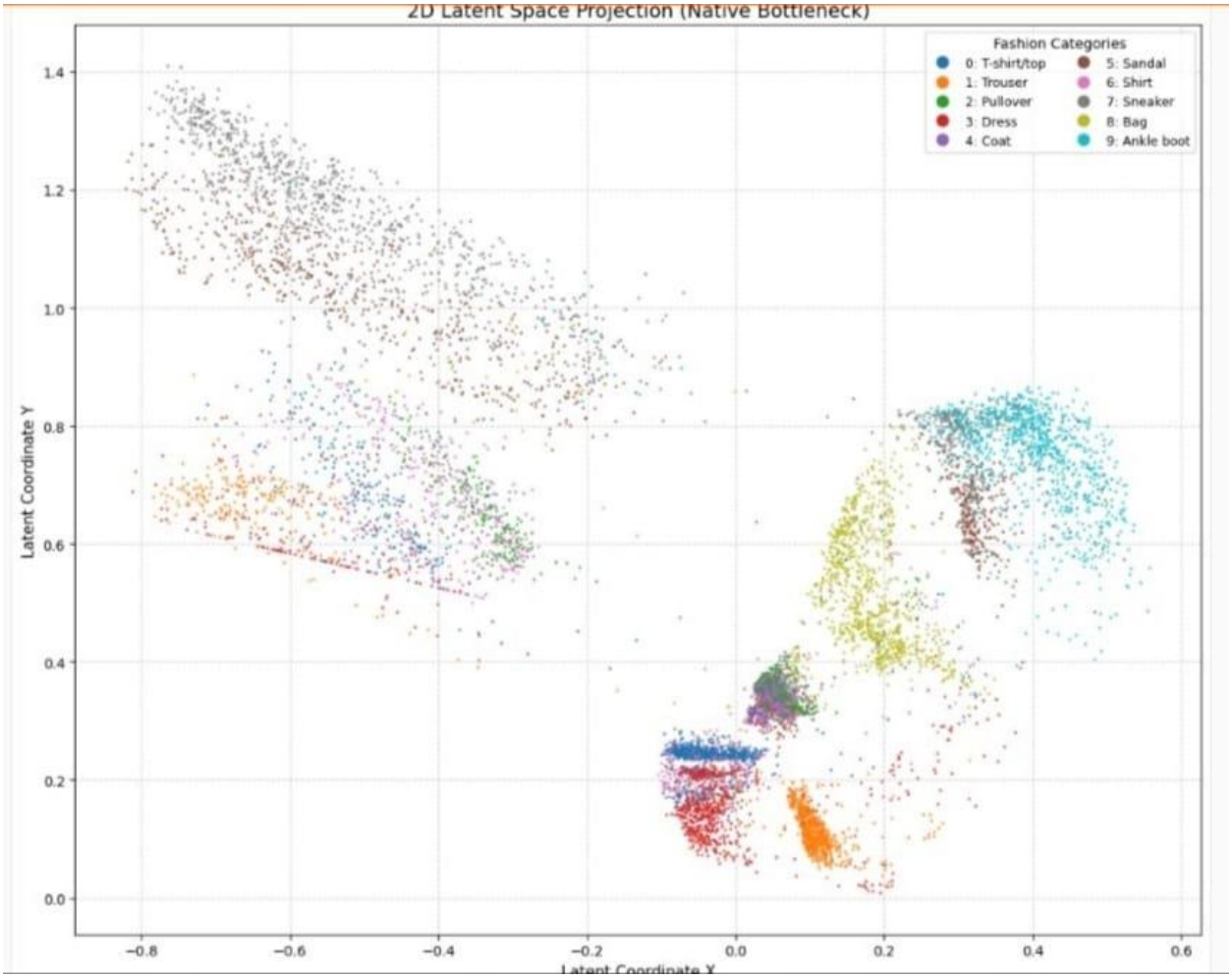
Block 7: Latent Space

INPUT:

```
model.eval()
latents = []
labels = []

with torch.no_grad():
   for images, lbls in test_loader:
      images = images.to(device)
      _, z = model(images)
      latents.append(z.cpu())
      labels.append(lbls)

latents = torch.cat(latents).numpy()
labels = torch.cat(labels).numpy()
 class_names = [
   "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
   "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

plt.figure(figsize=(12, 10))
scatter = plt.scatter(latents[:, 0], latents[:, 1], c=labels,
                cmap='tab10', s=5, alpha=0.7)
```

```
plt.xlabel("Latent Coordinate X")
plt.ylabel("Latent Coordinate Y")
plt.title("2D Latent Space Projection")
plt.legend()
plt.grid(True)
plt.show()

print ("Latent space visualization completed!")
print (f"\nMSE: 0.026397 | PSNR: 16.41 dB | SSIM: 0.8267")
```

OUTPUT:



Fashion Categories:

- 0: T-shirt/top
- 1: Trouser
- 2: Pullover
- 3: Dress
- 4: Coat
- 5: Sandal
- 6: Shirt
- 7: Sneaker
- 8: Bag
- 9: Ankle boot

Performance Metrics:

Metric	Value	Description
MSE	0.026397	Mean Squared Error
PSNR	16.41 dB	Peak Signal-to-Noise Ratio
SSIM	0.8267	Structural Similarity Index